

CSA0613 DESIGN ANALYSIS OF ALGORITHMS

Name: Nivasini VM

Reg no:192511356

CO2_ASSESSMENT TOOL 2

Q26. Debugging Convex Hull Logic (Incorrect Inclusion of Interior Points)

The given brute force Convex Hull logic incorrectly accepts some interior edges as hull edges

because the validation condition is incomplete. Analyze the hull-checking logic and identify

why non-boundary segments are being included. Apply systematic debugging to ensure that

only valid outer boundary edges are selected. Optimize the checking process by avoiding

unnecessary repeated evaluations where possible. Analyze the time complexity of the corrected brute force solution. Present a clean and structured version of the improved logic.

```
def is_hull_edge(p1, p2, points):
```

```
    for p in points:
```

```
        val = orientation(p1, p2, p)
```

```
        if val < 0:
```

```
            return False
```

```
    return True
```

Aim

To debug the brute force Convex Hull algorithm by correcting the hull edge validation logic so that only the outer boundary edges are included while excluding interior edges.

Algorithm

1. Read the set of points.
2. Select every pair of points as a possible edge.
3. For each candidate edge:
 - Calculate the orientation of every other point.
 - Count whether points lie on the left side, right side, or exactly on the line.
4. If points exist on both sides of the line, reject the edge.
5. Otherwise, accept it as a Convex Hull edge.
6. Repeat until all point pairs are checked.
7. Display all valid hull edges.

Python Code

```
1 def orientation(p, q, r):
2     return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
3 def is_hull_edge(p1, p2, points):
4     pos = 0
5     neg = 0
6     for p in points:
7         if p == p1 or p == p2:
8             continue
9         val = orientation(p1, p2, p)
10        if val > 0:
11            pos += 1
12        elif val < 0:
13            neg += 1
14        if pos > 0 and neg > 0:
15            return False
16    return True
17 points = [(0,0), (0,3), (3,0), (3,3), (1,1), (2,2)]
18 print("Convex Hull Edges:")
19 for i in range(len(points)):
20     for j in range(i+1, len(points)):
21         if is_hull_edge(points[i], points[j], points):
22             print(points[i], "-", points[j])
```

Output

```
Convex Hull Edges:  
(0, 0) - (0, 3)  
(0, 0) - (3, 0)  
(0, 3) - (3, 3)  
(3, 0) - (3, 3)  
  
=== Code Execution Successful ===
```

Result

The Convex Hull algorithm was successfully debugged. The corrected validation checks whether all remaining points lie on only one side of a candidate edge, ensuring that interior edges are rejected and only true hull edges are selected.

Analysis

Problem in the Original Logic

Original code:

```
def is_hull_edge(p1, p2, points):  
    for p in points:  
        val = orientation(p1, p2, p)  
        if val < 0:  
            return False  
    return True
```

Issues:

- It checks only if any point lies on one particular side ($val < 0$).
- It assumes the correct hull orientation beforehand.
- If all points happen to lie on the opposite side, the edge is accepted even if it is an interior edge.
- Collinear cases are not handled properly.
- The function may incorrectly classify internal segments as hull edges.

Debugging Improvements

- Skip the two endpoints while checking.
- Count points on both positive and negative sides.
- Reject an edge immediately if points appear on both sides.
- Accept the edge only if all remaining points lie on one side or are collinear.
- Early termination reduces unnecessary computations.

Time Complexity

Orientation Calculation

Each orientation check takes:

$O(1)$

Hull Edge Checking

For every pair of points, all remaining points are checked.

- Number of point pairs = $O(n^2)$
- Checking all points = $O(n)$

Overall Time Complexity:

$O(n^3)$

Space Complexity

Only a few variables (pos, neg, val) are used.

Space Complexity = $O(1)$

Optimization Used

- Early Exit: As soon as points are found on both sides of the candidate edge, the algorithm stops checking that edge.
- Skipping Endpoints: The selected edge endpoints are not re-evaluated.
- Avoids unnecessary repeated orientation checks after an invalid edge is detected.

These optimizations improve practical performance while keeping the worst-case time complexity at $O(n^3)$.